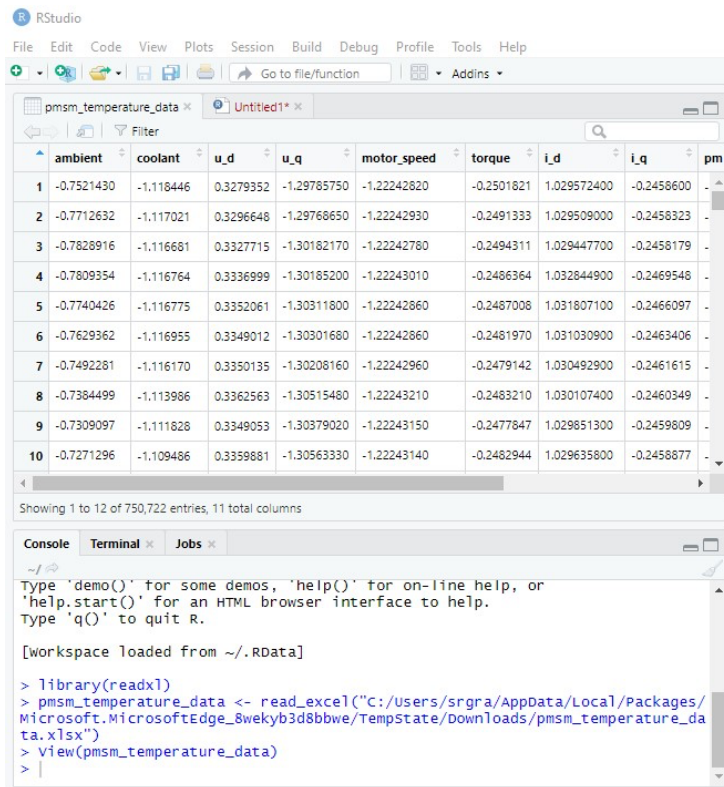


Final Project

Sean Graham

1.



RStudio interface showing a data frame with 11 columns and 10 rows of data. The columns are: ambient, coolant, u_d, u_q, motor_speed, torque, i_d, i_q, and pm. The data is displayed in a table view.

	ambient	coolant	u_d	u_q	motor_speed	torque	i_d	i_q	pm
1	-0.7521430	-1.118446	0.3279352	-1.29785750	-1.22242820	-0.2501821	1.029572400	-0.2458600	-
2	-0.7712632	-1.117021	0.3296648	-1.29768650	-1.22242930	-0.2491333	1.029509000	-0.2458323	-
3	-0.7828916	-1.116681	0.3327715	-1.30182170	-1.22242780	-0.2494311	1.029447700	-0.2458179	-
4	-0.7809354	-1.116764	0.3336999	-1.30185200	-1.22243010	-0.2486364	1.032844900	-0.2469548	-
5	-0.7740426	-1.116775	0.3352061	-1.30311800	-1.22242860	-0.2487008	1.031807100	-0.2466097	-
6	-0.7629362	-1.116955	0.3349012	-1.30301680	-1.22242860	-0.2481970	1.031030900	-0.2463406	-
7	-0.7492281	-1.116170	0.3350135	-1.30208160	-1.22242960	-0.2479142	1.030492900	-0.2461615	-
8	-0.7384499	-1.113986	0.3362563	-1.30515480	-1.22243210	-0.2483210	1.030107400	-0.2460349	-
9	-0.7309097	-1.111828	0.3349053	-1.30379020	-1.22243150	-0.2477847	1.029851300	-0.2459809	-
10	-0.7271296	-1.109486	0.3359881	-1.30563330	-1.22243140	-0.2482944	1.029635800	-0.2458877	-

Showing 1 to 12 of 750,722 entries, 11 total columns

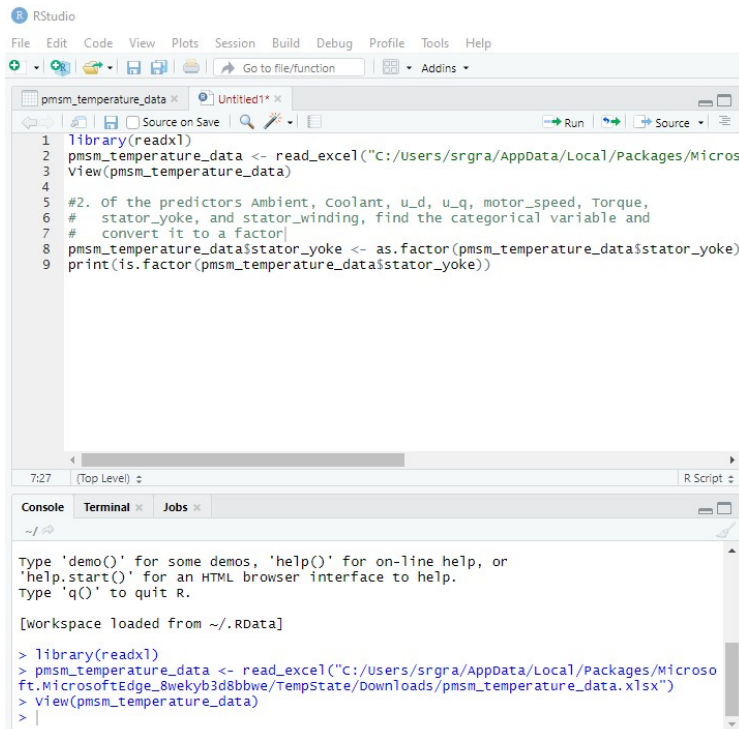
Console

```
~/
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[workspace loaded from ~/.RData]

> library(readxl)
> pmsm_temperature_data <- read_excel("C:/Users/srgra/AppData/Local/Packages/MicrosoftEdge_8wekyb3d8bbwe/TempState/Downloads/pmsm_temperature_data.xlsx")
> view(pmsm_temperature_data)
> |
```

2. The categorical variable is stator_yoke.



RStudio interface showing R code to read and view data, and convert a variable to a factor.

```
1 library(readxl)
2 pmsm_temperature_data <- read_excel("C:/Users/srgra/AppData/Local/Packages/MicrosoftEdge_8wekyb3d8bbwe/TempState/Downloads/pmsm_temperature_data.xlsx")
3 view(pmsm_temperature_data)
4
5 #2. of the predictors Ambient, Coolant, u_d, u_q, motor_speed, Torque,
6 # stator_yoke, and stator_winding, find the categorical variable and
7 # convert it to a factor
8 pmsm_temperature_data$stator_yoke <- as.factor(pmsm_temperature_data$stator_yoke)
9 print(is.factor(pmsm_temperature_data$stator_yoke))
```

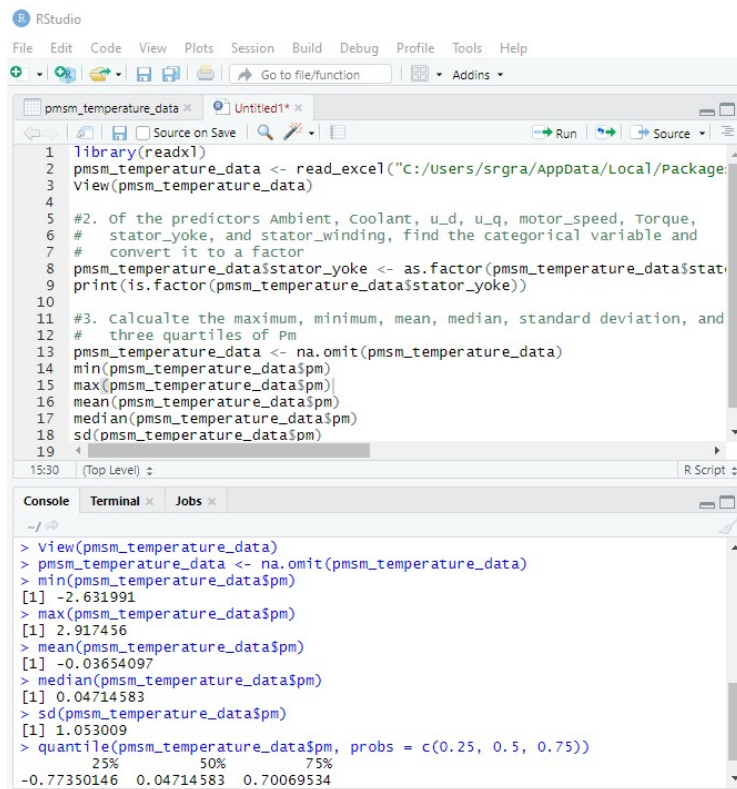
Console

```
~/
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[workspace loaded from ~/.RData]

> library(readxl)
> pmsm_temperature_data <- read_excel("C:/Users/srgra/AppData/Local/Packages/MicrosoftEdge_8wekyb3d8bbwe/TempState/Downloads/pmsm_temperature_data.xlsx")
> view(pmsm_temperature_data)
> |
```

3.



RStudio interface showing the following R code in the editor:

```

1 library(readxl)
2 pmsm_temperature_data <- read_excel("c:/users/srgra/AppData/Local/Package
3 View(pmsm_temperature_data)
4
5 #2. Of the predictors Ambient, Coolant, u_d, u_q, motor_speed, Torque,
6 # stator_yoke, and stator_winding, find the categorical variable and
7 # convert it to a factor
8 pmsm_temperature_data$stator_yoke <- as.factor(pmsm_temperature_data$stat
9 print(is.factor(pmsm_temperature_data$stator_yoke))
10
11 #3. Calculate the maximum, minimum, mean, median, standard deviation, and
12 # three quartiles of Pm
13 pmsm_temperature_data <- na.omit(pmsm_temperature_data)
14 min(pmsm_temperature_data$pm)
15 max(pmsm_temperature_data$pm)
16 mean(pmsm_temperature_data$pm)
17 median(pmsm_temperature_data$pm)
18 sd(pmsm_temperature_data$pm)
19

```

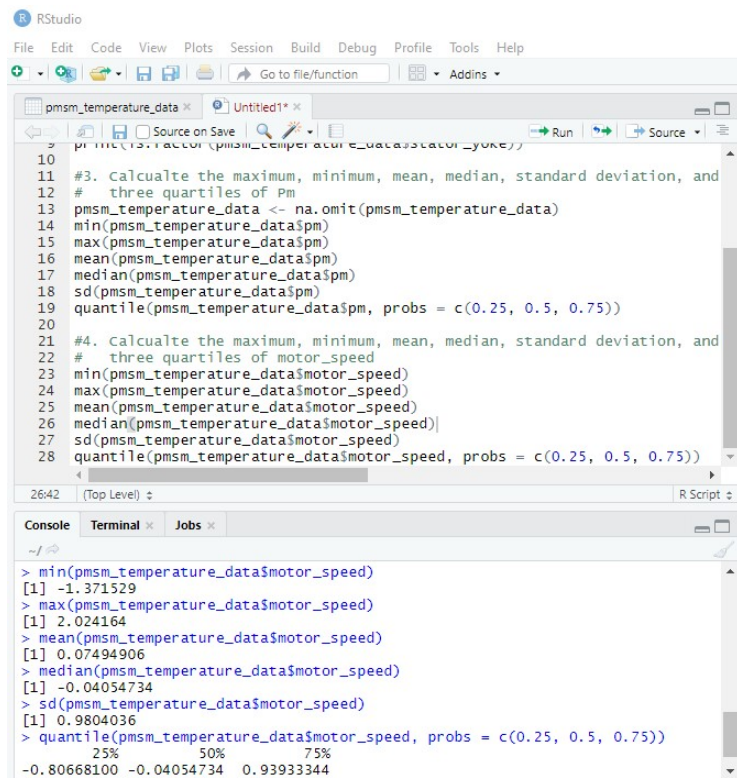
The console output shows the results of the executed code:

```

> View(pmsm_temperature_data)
> pmsm_temperature_data <- na.omit(pmsm_temperature_data)
> min(pmsm_temperature_data$pm)
[1] -2.631991
> max(pmsm_temperature_data$pm)
[1] 2.917456
> mean(pmsm_temperature_data$pm)
[1] -0.03654097
> median(pmsm_temperature_data$pm)
[1] 0.04714583
> sd(pmsm_temperature_data$pm)
[1] 1.053009
> quantile(pmsm_temperature_data$pm, probs = c(0.25, 0.5, 0.75))
      25%      50%      75%
-0.77350146  0.04714583  0.70069534

```

4.



RStudio interface showing the following R code in the editor:

```

10
11 #3. Calculate the maximum, minimum, mean, median, standard deviation, and
12 # three quartiles of Pm
13 pmsm_temperature_data <- na.omit(pmsm_temperature_data)
14 min(pmsm_temperature_data$pm)
15 max(pmsm_temperature_data$pm)
16 mean(pmsm_temperature_data$pm)
17 median(pmsm_temperature_data$pm)
18 sd(pmsm_temperature_data$pm)
19 quantile(pmsm_temperature_data$pm, probs = c(0.25, 0.5, 0.75))
20
21 #4. Calculate the maximum, minimum, mean, median, standard deviation, and
22 # three quartiles of motor_speed
23 min(pmsm_temperature_data$motor_speed)
24 max(pmsm_temperature_data$motor_speed)
25 mean(pmsm_temperature_data$motor_speed)
26 median(pmsm_temperature_data$motor_speed)
27 sd(pmsm_temperature_data$motor_speed)
28 quantile(pmsm_temperature_data$motor_speed, probs = c(0.25, 0.5, 0.75))

```

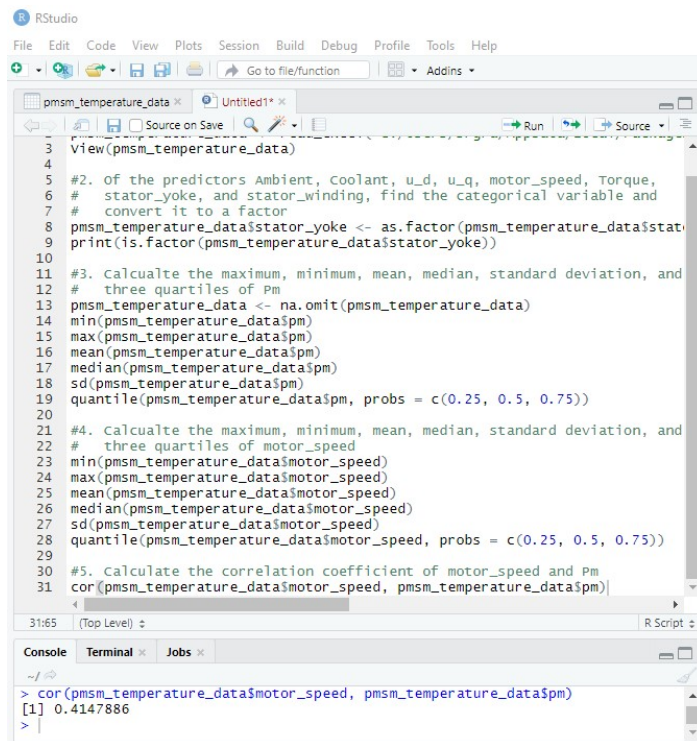
The console output shows the results of the executed code:

```

> min(pmsm_temperature_data$motor_speed)
[1] -1.371529
> max(pmsm_temperature_data$motor_speed)
[1] 2.024164
> mean(pmsm_temperature_data$motor_speed)
[1] 0.07494906
> median(pmsm_temperature_data$motor_speed)
[1] -0.04054734
> sd(pmsm_temperature_data$motor_speed)
[1] 0.9804036
> quantile(pmsm_temperature_data$motor_speed, probs = c(0.25, 0.5, 0.75))
      25%      50%      75%
-0.80668100 -0.04054734  0.93933344

```

5. Pm and motor_speed don't have a very strong relationship; the correlation coefficient is only about 0.415.



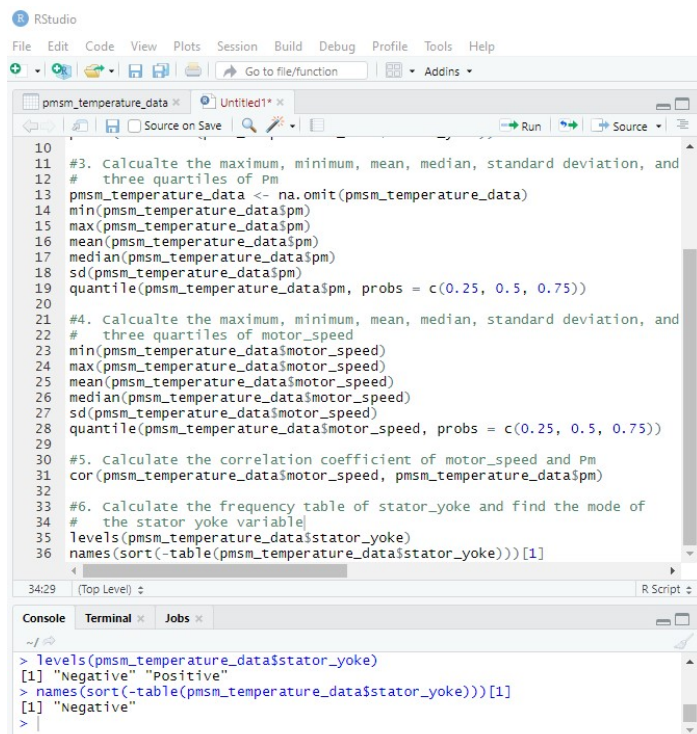
```
1 View(pmsm_temperature_data)
2
3 #2. Of the predictors Ambient, Coolant, u_d, u_q, motor_speed, Torque,
4 # stator_yoke, and stator_winding, find the categorical variable and
5 # convert it to a factor
6 pmsm_temperature_data$stator_yoke <- as.factor(pmsm_temperature_data$stator_yoke)
7 print(is.factor(pmsm_temperature_data$stator_yoke))
8
9 #3. Calculate the maximum, minimum, mean, median, standard deviation, and
10 # three quartiles of Pm
11 pmsm_temperature_data <- na.omit(pmsm_temperature_data)
12 min(pmsm_temperature_data$pm)
13 max(pmsm_temperature_data$pm)
14 mean(pmsm_temperature_data$pm)
15 median(pmsm_temperature_data$pm)
16 sd(pmsm_temperature_data$pm)
17 quantile(pmsm_temperature_data$pm, probs = c(0.25, 0.5, 0.75))
18
19 #4. Calculate the maximum, minimum, mean, median, standard deviation, and
20 # three quartiles of motor_speed
21 min(pmsm_temperature_data$motor_speed)
22 max(pmsm_temperature_data$motor_speed)
23 mean(pmsm_temperature_data$motor_speed)
24 median(pmsm_temperature_data$motor_speed)
25 sd(pmsm_temperature_data$motor_speed)
26 quantile(pmsm_temperature_data$motor_speed, probs = c(0.25, 0.5, 0.75))
27
28 #5. Calculate the correlation coefficient of motor_speed and Pm
29 cor(pmsm_temperature_data$motor_speed, pmsm_temperature_data$pm)
30
```

31:65 (Top Level) R Script

Console Terminal Jobs

```
> cor(pmsm_temperature_data$motor_speed, pmsm_temperature_data$pm)
[1] 0.4147886
>
```

6. The mode of stator_yoke is "negative."



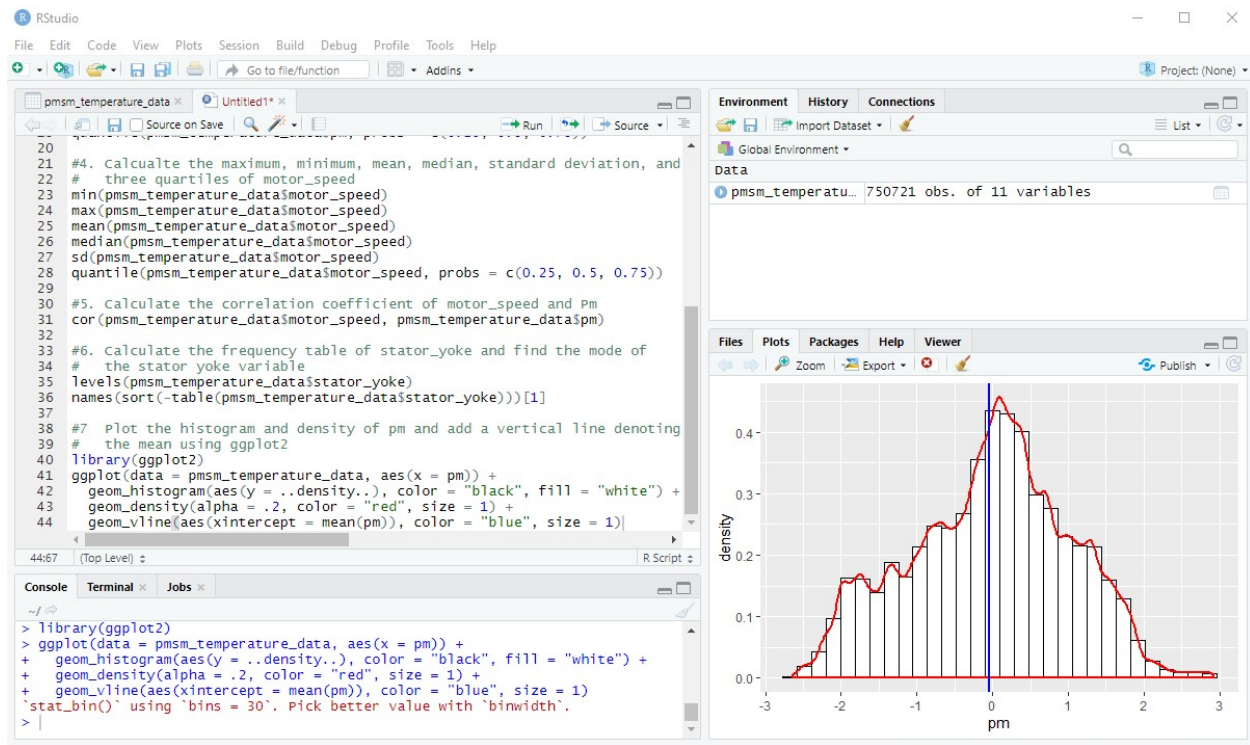
```
10 #3. Calculate the maximum, minimum, mean, median, standard deviation, and
11 # three quartiles of Pm
12 pmsm_temperature_data <- na.omit(pmsm_temperature_data)
13 min(pmsm_temperature_data$pm)
14 max(pmsm_temperature_data$pm)
15 mean(pmsm_temperature_data$pm)
16 median(pmsm_temperature_data$pm)
17 sd(pmsm_temperature_data$pm)
18 quantile(pmsm_temperature_data$pm, probs = c(0.25, 0.5, 0.75))
19
20 #4. Calculate the maximum, minimum, mean, median, standard deviation, and
21 # three quartiles of motor_speed
22 min(pmsm_temperature_data$motor_speed)
23 max(pmsm_temperature_data$motor_speed)
24 mean(pmsm_temperature_data$motor_speed)
25 median(pmsm_temperature_data$motor_speed)
26 sd(pmsm_temperature_data$motor_speed)
27 quantile(pmsm_temperature_data$motor_speed, probs = c(0.25, 0.5, 0.75))
28
29 #5. Calculate the correlation coefficient of motor_speed and Pm
30 cor(pmsm_temperature_data$motor_speed, pmsm_temperature_data$pm)
31
32 #6. Calculate the frequency table of stator_yoke and find the mode of
33 # the stator yoke variable
34 levels(pmsm_temperature_data$stator_yoke)
35 names(sort(-table(pmsm_temperature_data$stator_yoke)))[1]
36
```

34:29 (Top Level) R Script

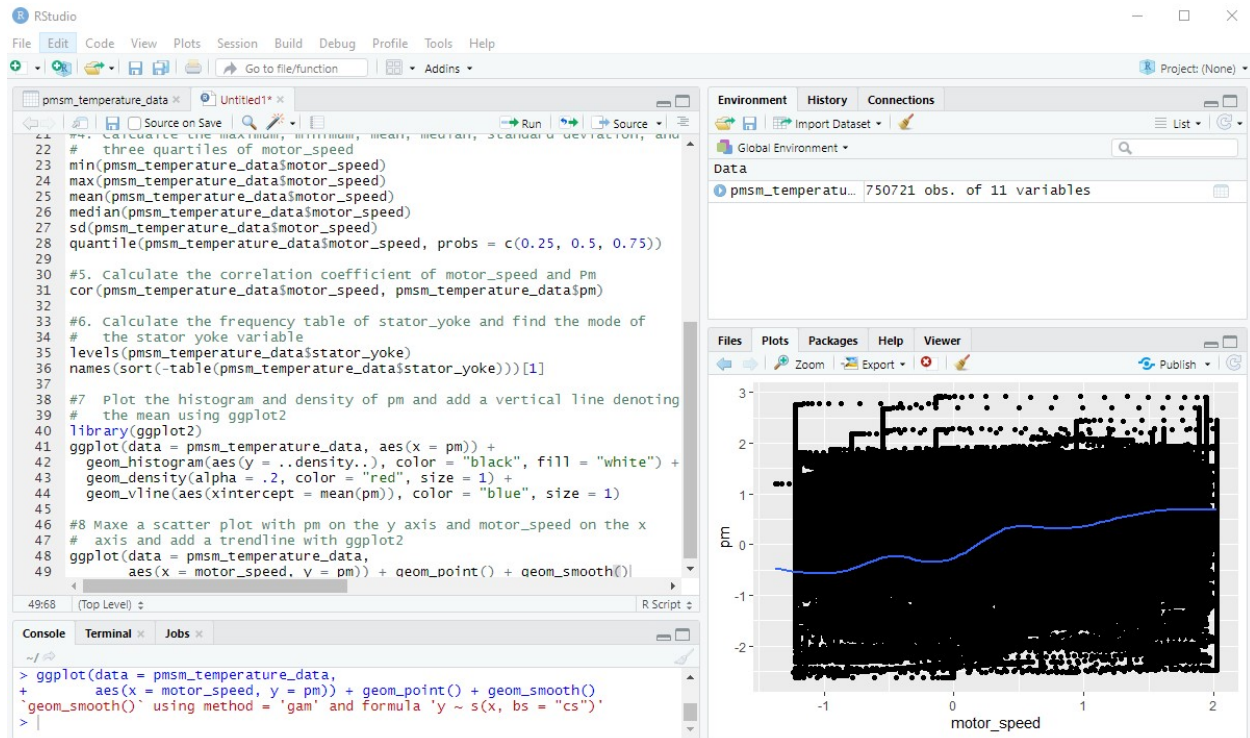
Console Terminal Jobs

```
> levels(pmsm_temperature_data$stator_yoke)
[1] "Negative" "Positive"
> names(sort(-table(pmsm_temperature_data$stator_yoke)))[1]
[1] "Negative"
>
```

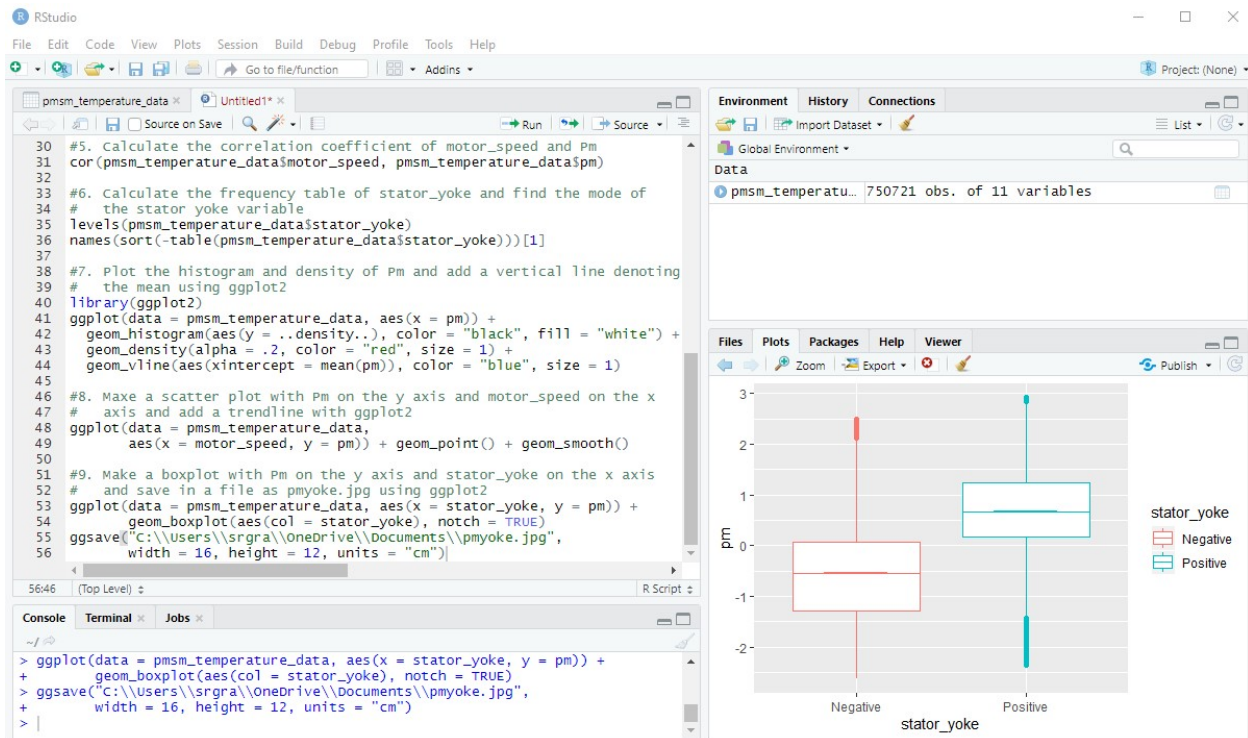

7.



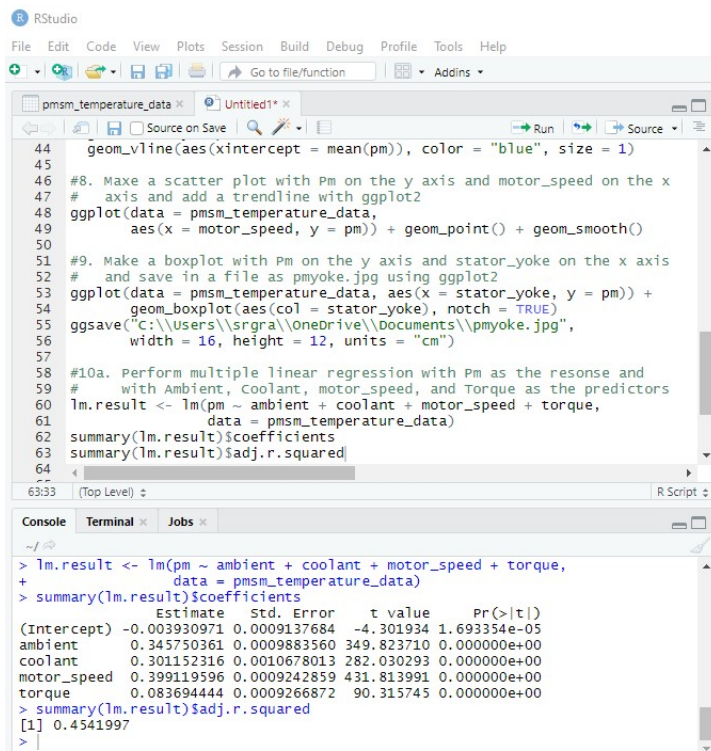
8.



9. Pm is generally larger when stator_yoke is positive. It should be noted that the notches are very small, but they're there.



10A. The formula is: $-0.0039 + 0.3458 * \text{Ambient} + 0.3012 * \text{Coolant} + 0.3991 * \text{motor_speed} + 0.0837 * \text{Torque}$.



10B. The formula is: $-0.0066 + 0.2801 * \text{Ambient} + -0.0107 * \text{Coolant} + -0.2700 * u_d + 0.0540 * \text{motor_speed} + -0.2482 * \text{Torque} + 0.6168 * \text{stator_winding}$.

```

54 geom_boxplot(aes(col = stator_yoke), notch = TRUE)
55 ggsave("C:\\Users\\largo\\OneDrive\\Documents\\pnyoke.jpg",
56        width = 16, height = 12, units = "cm")
57
58 #10a. Perform multiple linear regression with Pm as the response and
59 # with Ambient, coolant, motor_speed, and torque as the predictors
60 lm.result1 <- lm(Pm ~ ambient + coolant + motor_speed + torque,
61                 data = pmsm_temperature_data)
62 summary(lm.result1)$coefficients
63 summary(lm.result1)$adj.r.squared
64
65 #10b. Perform multiple linear regression with Pm as the response and
66 # with Ambient, coolant, u_d, motor_speed, Torque, and stator_winding
67 # as the predictors
68 lm.result2 <- lm(Pm ~ ambient + coolant + u_d + motor_speed + torque
69                 + stator_winding, data = pmsm_temperature_data)
70 summary(lm.result2)$coefficients
71 summary(lm.result2)$adj.r.squared

```

```

> lm.result2 <- lm(Pm ~ ambient + coolant + u_d + motor_speed + torque
+                  + stator_winding, data = pmsm_temperature_data)
> summary(lm.result2)$coefficients
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.006636084 0.0007063755  -9.394337 1.761186e-21
ambient      0.280087056 0.0007690711  363.763001 0.000000e+00
coolant      -0.010662764 0.0009510803  -11.211213 3.611612e-29
u_d          -0.270032724 0.0013021679 -207.371666 0.000000e+00
motor_speed   0.054033292 0.0008933072   60.492737 0.000000e+00
torque       -0.248202777 0.0011786538  -212.270703 0.000000e+00
stator_winding 0.616822296 0.0009055660  681.145584 0.000000e+00
> summary(lm.result2)$adj.r.squared
[1] 0.6738776

```

10C. The formula is a piecewise function.

When stator_yoke is positive, the formula is: $0.1243 + 0.2831 * \text{Ambient} + -0.0516 * \text{Coolant} + -0.2515 * u_d + -0.0601 * u_q + 0.1142 * \text{motor_speed} + -0.2605 * \text{Torque} + 0.5369 * \text{stator_winding}$.

When stator_yoke is negative, the formula is: $-0.1192 + 0.2831 * \text{Ambient} + -0.0516 * \text{Coolant} + -0.2515 * u_d + -0.0601 * u_q + 0.1142 * \text{motor_speed} + -0.2605 * \text{Torque} + 0.5369 * \text{stator_winding}$.

```

66 # with Ambient, coolant, u_d, motor_speed, Torque, and stator_winding
67 # as the predictors
68 lm.result2 <- lm(Pm ~ ambient + coolant + u_d + motor_speed + torque
69                 + stator_winding, data = pmsm_temperature_data)
70 summary(lm.result2)$coefficients
71 summary(lm.result2)$adj.r.squared
72
73 #10c. Perform multiple linear regression with Pm as the response and
74 # with Ambient, coolant, u_d, u_q, motor_speed, Torque, stator_yoke,
75 # and stator_winding as the predictors
76 lm.result3 <- lm(Pm ~ ambient + coolant + u_d + u_q + motor_speed +
77                 torque + stator_yoke + stator_winding,
78                 data = pmsm_temperature_data)
79 summary(lm.result3)$coefficients
80 summary(lm.result3)$adj.r.squared

```

```

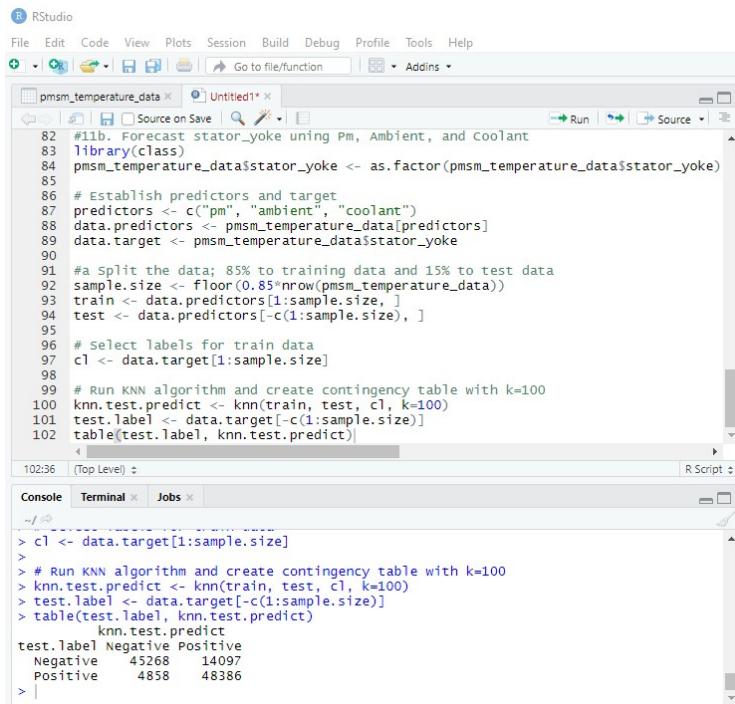
> lm.result3 <- lm(Pm ~ ambient + coolant + u_d + u_q + motor_speed +
+                  torque + stator_yoke + stator_winding,
+                  data = pmsm_temperature_data)
> summary(lm.result3)$coefficients
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -0.11915072 0.0013486324  -88.34929    0
ambient      0.28312684 0.0007651849  370.01100    0
coolant      -0.05159457 0.0011291343  -45.69392    0
u_d          -0.25151648 0.0013442833 -187.10080    0
u_q          -0.06007881 0.0010906638  -55.08463    0
motor_speed   0.11421214 0.0013311363   85.80048    0
torque       -0.26052010 0.0013136094 -198.32387    0
stator_yokePositive 0.24344746 0.0024341920  100.01161    0
stator_winding 0.53693677 0.0011552027  464.79877    0
> summary(lm.result3)$adj.r.squared
[1] 0.6791595

```


10D. The adjusted R squared values for A, B, and C are 0.454, 0.674, and 0.679. I would choose model C; its value is closest to one, making it the best fit.

11A. This is performed within parts B, C, and D. See the images below.

11B.



```
#11b. Forecast stator_yoke using pm, Ambient, and Coolant
library(class)
pmsm_temperature_data$stator_yoke <- as.factor(pmsm_temperature_data$stator_yoke)

# Establish predictors and target
predictors <- c("pm", "ambient", "coolant")
data.predictors <- pmsm_temperature_data[predictors]
data.target <- pmsm_temperature_data$stator_yoke

# Split the data; 85% to training data and 15% to test data
sample.size <- floor(0.85*nrow(pmsm_temperature_data))
train <- data.predictors[1:sample.size, ]
test <- data.predictors[-c(1:sample.size), ]

# Select labels for train data
cl <- data.target[1:sample.size]

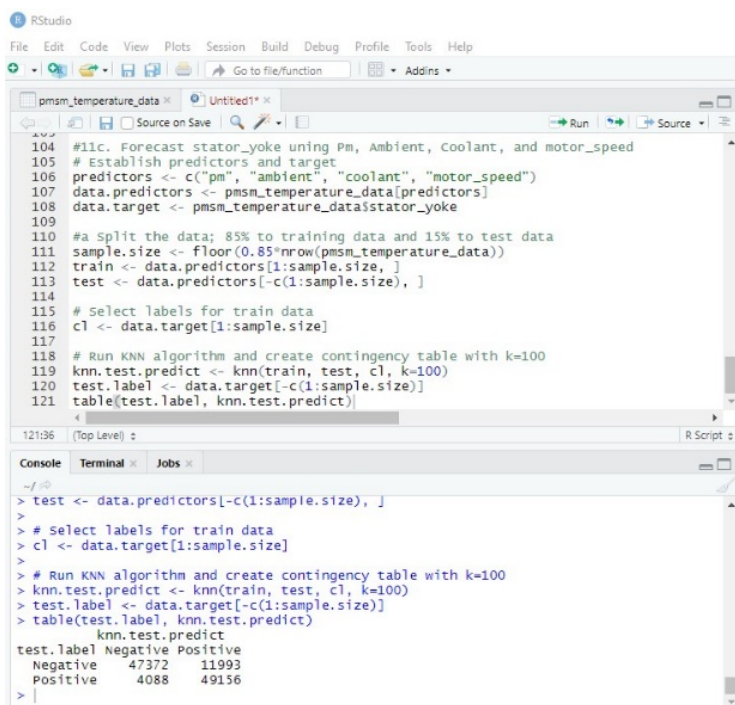
# Run KNN algorithm and create contingency table with k=100
knn.test.predict <- knn(train, test, cl, k=100)
test.label <- data.target[-c(1:sample.size)]
table(test.label, knn.test.predict)
```

102:36 (Top Level) R Script

Console

```
> cl <- data.target[1:sample.size]
> 
> # Run KNN algorithm and create contingency table with k=100
> knn.test.predict <- knn(train, test, cl, k=100)
> test.label <- data.target[-c(1:sample.size)]
> table(test.label, knn.test.predict)
      knn.test.predict
test.label Negative Positive
Negative    45268    14097
Positive    4858    48386
>
```

11C.



```
#11c. Forecast stator_yoke using pm, Ambient, Coolant, and motor_speed
# Establish predictors and target
predictors <- c("pm", "ambient", "coolant", "motor_speed")
data.predictors <- pmsm_temperature_data[predictors]
data.target <- pmsm_temperature_data$stator_yoke

# Split the data; 85% to training data and 15% to test data
sample.size <- floor(0.85*nrow(pmsm_temperature_data))
train <- data.predictors[1:sample.size, ]
test <- data.predictors[-c(1:sample.size), ]

# Select labels for train data
cl <- data.target[1:sample.size]

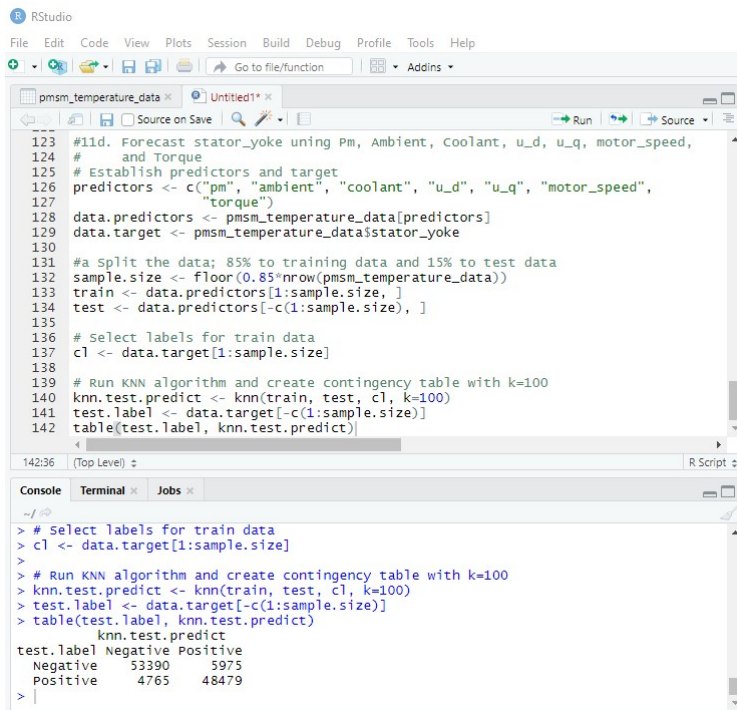
# Run KNN algorithm and create contingency table with k=100
knn.test.predict <- knn(train, test, cl, k=100)
test.label <- data.target[-c(1:sample.size)]
table(test.label, knn.test.predict)
```

121:36 (Top Level) R Script

Console

```
> test <- data.predictors[-c(1:sample.size), ]
> 
> # Select labels for train data
> cl <- data.target[1:sample.size]
> 
> # Run KNN algorithm and create contingency table with k=100
> knn.test.predict <- knn(train, test, cl, k=100)
> test.label <- data.target[-c(1:sample.size)]
> table(test.label, knn.test.predict)
      knn.test.predict
test.label Negative Positive
Negative    47372    11993
Positive    4088    49156
>
```

11D.



```
123 #11d. Forecast stator_yoke using Pm, Ambient, Coolant, u_d, u_q, motor_speed,
124 # and Torque
125 # Establish predictors and target
126 predictors <- c("pm", "ambient", "coolant", "u_d", "u_q", "motor_speed",
127 "torque")
128 data.predictors <- pmsm_temperature_data[predictors]
129 data.target <- pmsm_temperature_data$stator_yoke
130
131 #a Split the data; 85% to training data and 15% to test data
132 sample.size <- floor(0.85*nrow(pmsm_temperature_data))
133 train <- data.predictors[1:sample.size, ]
134 test <- data.predictors[-c(1:sample.size), ]
135
136 # Select labels for train data
137 cl <- data.target[1:sample.size]
138
139 # Run KNN algorithm and create contingency table with k=100
140 knn.test.predict <- knn(train, test, cl, k=100)
141 test.label <- data.target[-c(1:sample.size)]
142 table(test.label, knn.test.predict)
```

142:36 (Top Level) R Script

Console Terminal Jobs

```
> # select labels for train data
> cl <- data.target[1:sample.size]
>
> # Run KNN algorithm and create contingency table with k=100
> knn.test.predict <- knn(train, test, cl, k=100)
> test.label <- data.target[-c(1:sample.size)]
> table(test.label, knn.test.predict)
```

	knn.test.predict	
test.label	Negative	Positive
Negative	53390	5975
Positive	4765	48479

11E. Accuracy of B: $(45268 + 48386) / (45268 + 48386 + 14097 + 4858) = 83.167\%$

Accuracy of C: $(47372 + 49156) / (47372 + 49156 + 11993 + 4088) = 85.792\%$

Accuracy of D: $(53390 + 48479) / (53390 + 48479 + 5975 + 4765) = 90.463\%$

The model in part D has the highest accuracy, so this would be the best one to use.